

# MÓDULO VII

## Introdução à Programação II

# MÓDULO VII

## Entendendo o Tratamento de Exceções

O tratamento de exceções é um recurso fundamental em muitas linguagens de programação, incluindo **JavaScript**, que permite ao desenvolvedor gerenciar erros de forma controlada. Em **JavaScript**, isso é realizado por meio dos blocos **try**, **catch**, **finally** e da instrução **throw**.

### O Bloco try

O bloco try é usado para envolver o código suscetível a erros. O JavaScript tenta executar o código dentro deste bloco, e se um erro ocorre, a execução é interrompida e o controle é transferido para o bloco catch mais próximo.

# MÓDULO VII

## O Bloco **catch**

Após a captura de um erro pelo bloco **try**, o bloco **catch** é executado, permitindo que o *erro* seja tratado de forma adequada. O bloco **catch** recebe um argumento, geralmente denominado *error*, que contém informações sobre o *erro* ocorrido.

## O Bloco **finally**

Opcionalmente, o bloco **finally** pode ser utilizado após os blocos **try** e **catch**. Este bloco será executado independentemente do resultado anterior, sendo ideal para realizar limpezas ou finalizar processos que devem ser concluídos após a tentativa de execução do código, seja ela bem-sucedida ou não.

## A Instrução **throw**

Para lançar uma exceção manualmente, utiliza-se a instrução **throw**. Esse recurso permite que o desenvolvedor crie condições específicas de *erro*, as quais podem ser capturadas e tratadas pelos blocos **catch**.

# MÓDULO VII

A declaração **try** consiste em um bloco **try**, que contém uma ou mais declarações, e ao menos uma cláusula **catch** ou uma cláusula **finally**, ou ambas. Ou seja, há 3 formas de declarações **try** :

1. **try...catch**
2. **try...finally**
3. **try...catch...finally**

Uma cláusula **catch** contém declarações que especificam o que fazer caso uma exceção seja lançada no bloco **try**. Ou seja, se você quer que o bloco **try** tenha êxito, e caso não tenha, você quer que o controle passe para o bloco **catch**. Caso qualquer declaração dentro do bloco **try** (*ou em uma função chamada no interior do bloco try*) lançar uma exceção o controle imediatamente muda para a cláusula **catch**. Se nenhuma exceção for lançada no bloco **try** a cláusula **catch** é ignorada.

# MÓDULO VII

A cláusula **finally** é executada após a execução do bloco **try** e da(s) cláusula(s) **catch** porém antes das declarações seguintes ao **try**. Ela sempre é executada, independente se uma exceção for lançada ou capturada.

Você pode aninhar uma ou mais declarações **try**. Caso uma declaração **try** interior não tenha uma cláusula **catch**, a cláusula **catch** pertencente a declaração **try** que a envolve é introduzida.

Você pode usar a declaração **try** para manipular exceções em **JavaScript**.

# MÓDULO VII

## Exemplo Prático:

```
function verificarIdade(idade) {  
  try {  
    // Suponha que exista uma regra que a idade deve ser maior que 18  
    if (idade < 18) {  
      // Lançando uma string como "erro"  
      throw "Desculpe, você deve ter mais de 18 anos.";  
    }  
  
    // Se a idade for maior ou igual a 18, essa mensagem será exibida  
    console.log("Idade verificada com sucesso.");  
  } catch (erro) {  
    // Capturando o "erro" lançado e exibindo a mensagem  
    console.log("Erro: " + erro);  
  } finally {  
    // Este bloco será executado independentemente do resultado  
    console.log("Verificação de idade concluída.");  
  }  
}  
  
verificarIdade(16)
```

# MÓDULO VII

No último exemplo, a instrução **throw** é usada para lançar um *erro* manualmente, que é então capturado pelo bloco **catch**. Independentemente do que aconteça nos blocos **try** e **catch**, o bloco **finally** será executado.

Este mecanismo de tratamento de exceções permite que você escreva aplicativos mais robustos e confiáveis, tratando erros de maneira controlada sem interromper completamente a execução do programa.

O objeto **Error** é uma peça fundamental no tratamento de exceções em **JavaScript**. Ele é utilizado para criar uma instância de erro que pode ser lançada com a instrução **throw** e posteriormente capturada por um bloco **catch**. O objeto **Error** não só contém a mensagem de *erro* como também outras propriedades úteis, como o nome do *erro* e a pilha de chamadas, que podem ser extremamente valiosas para depuração.

O tratamento de exceções em **JavaScript** é uma ferramenta poderosa que permite aos desenvolvedores gerenciar *erros* de maneira eficaz. Ao entender e utilizar os blocos **try**, **catch**, **finally**, a instrução **throw** e o objeto **Error**, é possível escrever códigos mais robustos e confiáveis, melhorando significativamente a qualidade das aplicações **JavaScript**.

# MÓDULO VII

## Sintaxe do Try Catch

A sintaxe do **try catch** é composta por dois blocos principais: O **try** onde o código é executado e o **catch**, onde você recebe via parâmetro na função, um objeto do tipo **Error**.

Com este objeto você pode aplicar o tratamento que for adequado. Veja o exemplo:

```
try {  
    // seu código aqui  
} catch (error) {  
    // tratamento de erro aqui  
}
```

Você também pode estender o try catch usando a cláusula finally. Este bloco será executado independente se houver ou não falha, ou seja, depois que o try ou catch executar, este bloco será acionado. Isto pode ser útil por exemplo, para fechar um arquivo que foi aberto para leitura, registrar algum log ou fechar alguma conexão.

Veja a sintaxe de exemplo.

```
try {  
    // seu código aqui  
} catch (error) {  
    // tratamento de erro aqui  
} finally {  
    // executa sempre  
}
```



# MÓDULO VII

## Objeto Error

Sempre que uma exceção é lançada dentro do bloco try, o JavaScript cria um objeto do tipo **Error** e envia como argumento para o catch. Por padrão, este objeto é composto de duas propriedades principais:

### **name:**

Representa o tipo do erro. Por exemplo, um erro de sintaxe no parse de um JSON, lançaria um exceção do tipo ***SyntaxError***.

### **message:**

Mensagem em texto, contendo mais detalhes do erro.

Em alguns ambientes onde você realiza o tratamento de erros em JavaScript, também terá a propriedade **stack**. Nela você tem a sequência de chamadas que levaram ao erro. Este tipo de informação é bastante útil para depuração.

# MÓDULO VII

Uma possibilidade adicional que você tem, é omitir o argumento **error** na função **catch**.

Isto é útil quando não precisamos saber os detalhes do **erro**, veja o exemplo:

```
try {  
    // seu código aqui  
} catch {  
    // tratamento de erro aqui sem o argumento  
}
```

Outra característica bem importante do objeto **Error** é que ele é a base de alguns tipos nativos.

O **JavaScript** possui os seguintes tipos de **erros** nativos que podem ser lançados:

## ReferenceError

Lançado quando uma referência a uma variável ou função inexistente ou inválida é detectada.

# MÓDULO VII

## **TypeError**

Lançado quando um **operador** ou **argumento** passado para a **função** é de um tipo diferente do esperado.

## **SyntaxError**

Lançado quando ocorre algum **erro** de sintaxe ao **interpretar** o código, por exemplo ao realizar o **parse** de um **JSON**.

## **URIError**

Lançado quando ocorre algum **erro** no tratamento de **URI**, por exemplo, enviando **parâmetros** inválidos no **decodeURI()** ou **encodeURI()**.

## **RangeError**

Lançado quando um valor não está no conjunto ou intervalo de valores permitidos. Por exemplo, um valor em **string** num **array** número.

Todos os **erros** nativos do **JavaScript**, são extensões do objeto **Error**, partindo deste princípio, você também pode criar seus próprios tipos de **erros**.

# MÓDULO VII

## Operador Throw

Se você analisou o exemplo de **erro** customizado, percebeu que usei o operador **throw** para lançar o **erro**. A principal função dele é basicamente esta.

Quando precisamos lançar um **exceção**, usamos a instrução **throw** precedida do valor, e este pode ser uma **string**, um **número**, um **objeto literal**, uma **função** ou até mesmo uma **classe**, como no exemplo anterior. Veja alguns exemplos de uso do Throw:

```
// Lançando uma string
throw "Descrição do erro"
// Lançando uma StatusCode HTTP
throw 404
// Lançando um objeto literal
throw {
  name: 'Erro',
  message: 'Descrição do erro'
}
// Lançando um erro customizado
throw new UserTypeError('Tipo de usuário inválido')
```

# MÓDULO VII

## Onde usar try catch

Este é um ponto bem interessante, talvez você já tenha captado uma abordagem que podemos adotar, ou até já tenha visto o uso de um **try catch global**, pegando toda a aplicação.

Uma boa prática é evitar um **try catch global**, o mais indicado é aplicar um **try catch** por função ou contextos da sua aplicação, para então tratar as exceções. Entenda, que uma vez chamada a exceção, seu script será interrompido.

# MÓDULO VII

## 1 try (Tentativa) – Verificar Wi-Fi

*Situação:* Tentar conectar a um Wi-Fi que pode falhar.

```
try {  
  console.log("Conectando ao Wi-Fi 'EscolaLegal'... 📶");  
  // Simulando falha (na vida real, seria uma API, etc.)  
  const wifiDisponivel = false;  
  if (!wifiDisponivel) throw new Error("Wi-Fi indisponível!");  
  console.log("Conectado!"); // Só executa se der certo  
} catch (erro) {  
  console.log("Falha: " + erro.message); // "Wi-Fi indisponível!"  
}
```

# MÓDULO VII

## 2 catch (Captura) – Senha do TikTok Errada

*Situação:* Digitar a senha errada em um app.

```
try {  
  const senhaCorreta = "123456";  
  const senhaDigitada = "abc123";  
  if (senhaDigitada !== senhaCorreta) throw new Error("Senha incorreta!");  
  console.log("Login feito!"); // Não chega aqui  
} catch (erro) {  
  console.log("Ops! " + erro.message); // "Senha incorreta!"  
  console.log("Dica: Tente sua data de nascimento."); // Ação p/erro  
}
```

# MÓDULO VII

## ③ **finally (Finalmente) – Bateria do Celular**

*Situação:* Mesmo que o celular desligue, ele mostra "Carregar!".

```
try {  
  console.log("Jogando Free Fire... 🔥");  
  throw new Error("Bateria acabou!"); // Desliga do nada  
} catch (erro) {  
  console.log("ALERTA: " + erro.message); // "Bateria acabou!"  
} finally {  
  console.log("👉 Conecte o carregador!"); // Sempre aparece  
}
```



# MÓDULO VII

## 4 throw (Lançar) – Idade para Ver Filme

*Situação:* Tentar entrar em um filme para maiores de 18 anos.

```
function verificarIdade(idade) {  
  if (idade < 18) throw new Error("Você é muito novo para este filme!");  
  console.log("Acesso permitido. Bom filme!"); // Só se idade >= 18  
}  
  
try {  
  verificarIdade(15); // Lança o erro!  
} catch (erro) {  
  console.log("ERRO: " + erro.message); // Exibe a mensagem  
}
```

# MÓDULO VII

## 5 Error – Nome de Usuário Inválido

**Situação:** Um jogo não deixa criar um nome de usuário muito curto.

```
// 1. Criando um tipo de erro personalizado
class NomeInvalidoError extends Error {
  constructor(mensagem) {
    super(mensagem);
    this.name = "NomeInvalidoError"; // Nome do erro
  }
}

// 2. Função que verifica o nome
function verificarNome(nome) {
  if (nome.length < 3) {
    throw new NomeInvalidoError("O nome precisa ter pelo menos 3 letras!");
  }
  console.log(`Nome "${nome}" válido! ✔`);
}

// 3. Testando
try {
  verificarNome("Ana"); // Funciona ✔
  verificarNome("Li"); // Erro ✖ (nome muito curto)
} catch (erro) {
  console.log("✖ " + erro.message); // Mostra a mensagem de erro
}
```

### Explicação Passo a Passo:

1. **NomeInvalidoError** → É um **tipo de erro novo** que criamos só para nomes inválidos.
2. **verificarNome()** → Checa se o nome tem **menos de 3 letras**. Se tiver, **lança o erro** (throw).
3. **try/catch** → Tenta executar, mas se der erro, **captura e mostra a mensagem**.

# MÓDULO VII

## 🎯 Versão ainda mais simples (sem classe de erro):

Se a classe **NomeInvalidoError** ainda parece complicada, pode usar só **throw new Error()**:

```
function verificarNome(nome) {  
  if (nome.length < 3) {  
    throw new Error("Nome muito curto! Use no mínimo 3 letras.");  
  }  
  console.log("Nome válido!");  
}  
  
try {  
  verificarNome("Jo"); // ❌ Vai dar erro!  
} catch (erro) {  
  console.log("ERRO: " + erro.message);  
}
```

📌 Resumo:

"**throw new Error()** é como gritar 'Isso não pode!' quando algo dá errado, e try/catch é ouvir esse grito e avisar o usuário."

# MÓDULO VII

## Função Verificar Senha (Versão Corrigida e Melhorada)

**Objetivo:** Impedir senhas óbvias como "123456".

```
function verificarSenha(senha) {  
  // Lista de senhas proibidas  
  const senhasFaceis = ["123456", "senha", "password", "123123", "qwerty"];  
  
  if (senhasFaceis.includes(senha)) {  
    throw new Error(`⚠️ Senha "${senha}" é muito fraca! Escolha outra.`);  
  }  
  console.log("✅ Senha segura! Pode usar.");  
}  
  
// Testando  
try {  
  verificarSenha("123456"); // Vai falhar ❌  
} catch (erro) {  
  console.log(erro.message); // Mostra o erro  
}  
  
try {  
  verificarSenha("minhasenha123"); // Vai passar ✅  
} catch (erro) {  
  console.log(erro.message);  
}
```

# MÓDULO VII

## EXERCÍCIOS...

# MÓDULO VII

FIM...